then you can create a virtual platform for any SoC. Such libraries of SystemC models are available as open source or commercial packages such as the system-level library.

SoC designs are dominated by ARM IP, so you will need models of ARM cores and bus sub-systems. ARM supplies such models by the name of Fast Models, and these models are already in wide use in virtual platforms worldwide.

You have seen how RTL can be used to replace missing transaction-level models. Using Standard Co-Emulation Modelling Interface (SCE-MI), you can create a hybrid emulation platform in order to substitute a transaction-level model for not-yet-available RTL blocks. In some cases, you might not have access to the RTL for the CPU IP. In such cases, you can either use Fast Models or mount a hardware test chip into the emulator. However, the latest CPU IP often has a virtual model available well before a customer-usable test chip.

SCE-MI can be used to solve the following customer-verification problem:

Most emulators on the market today offer some proprietary APIs in addition to SCE-MI 1.1 API.

Proliferation of APIs makes it very difficult for software based verification products to port to different emulators, thus restricting the solutions available to customers. This also leads to low productivity and low return on investment for emulator customers who build their own solutions.

Emulation APIs that exist today are oriented towards gate-level and not system-level verification.

If you cannot meet performance targets with the whole SoC in an FPGA based prototype or an emulator, and if you do not need cycle-accurate behaviour within the CPU, you can replace the CPU with Fast Models running in a virtual platform linked via SCE-MI.

Virtual models of the CPU run Instruction Set Simulation (ISS) rather than model the CPU's gate-level behaviour and, hence, perform much faster than running its RTL implemented in FPGA, or an emulator.

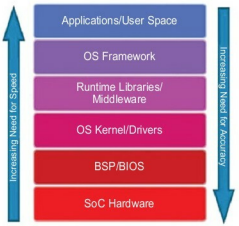

Fig. 7: Relative importance of speed and performance at different levels of a software stack

ARM Fast Models also represent additional device-specific functionalities such as cache coherency that a more generic ISS would ignore, but would still maintain the performance advantage. As an example, a CPU IP sub-system may typically run at 2GHz in 28nm silicon. But how fast will it run in various verification platforms?

There are many dependencies, but the CPU's RTL might only reach 20MHz when partitioned into an FPGA based prototype, or just 2MHz in a traditional Big Box emulator—an FPGA based emulator could be somewhere between those two figures. That same processor's virtual platform might run at a rate equivalent to more than 1GHz, although this figure is slightly misleading given that the models are untimed, so it is better considered in terms of operations per second.

The CPU or a CPU sub-system such as ARM Cortex-A53 cluster is usually delivered as pre-verified RTL, so re-verifying it fully to cycle-level accuracy may be a waste of time. All you need to do is to model the CPU's interfaces with the rest of the SoC with cycle-accuracy, while modelling the CPU's internal activity with untimed transactions. Operation of the CPU, its software and the overall SoC prototype will be accelerated as a result.

This use mode might also be valuable if the target FPGA hardware is running out of capacity such that you do not have room to implement all of the RTL in the FPGA hardware at once. By selectively pushing parts of the design over to the virtual side, you can not only boost performance but also free up FPGA resources.

In SoC verification, directed tests are often used in conjunction with constrained random tests in order to provide necessary functional coverage. The most sophisticated directed tests are written such that these can adapt the stimulus to reflect activity within the design under test. Such tests are often written in SystemC, so it is a short jump to consider driving these from a virtual platform running specific test software.

Verification teams often employ such software-driven tests, but this is usually assumed to be running on a processor embedded within the SoC hardware. Ability for software running on the virtual CPU to adapt stimulus to the response from the design under test is a powerful way to augment coverage, which may otherwise take a thousand times more verification cycles to reach by constrained random methods alone.

If you create a fix to some unexpected behaviour, you can test that fix in exactly the same combination of conditions that exposed the original bug. The hybrid solution drives the hardware with stimulus from the virtual side, for example, with communication messages and data in a certain protocol being extracted by the virtual model from data files on the virtual model's host workstation. A library of such stimulus might even be built up and reused across projects.

In the same way, virtual models can access real world data from a USB port on the FPGA based hardware, for example, in order to exercise middleware, and drivers being created by software engineers on the virtual